

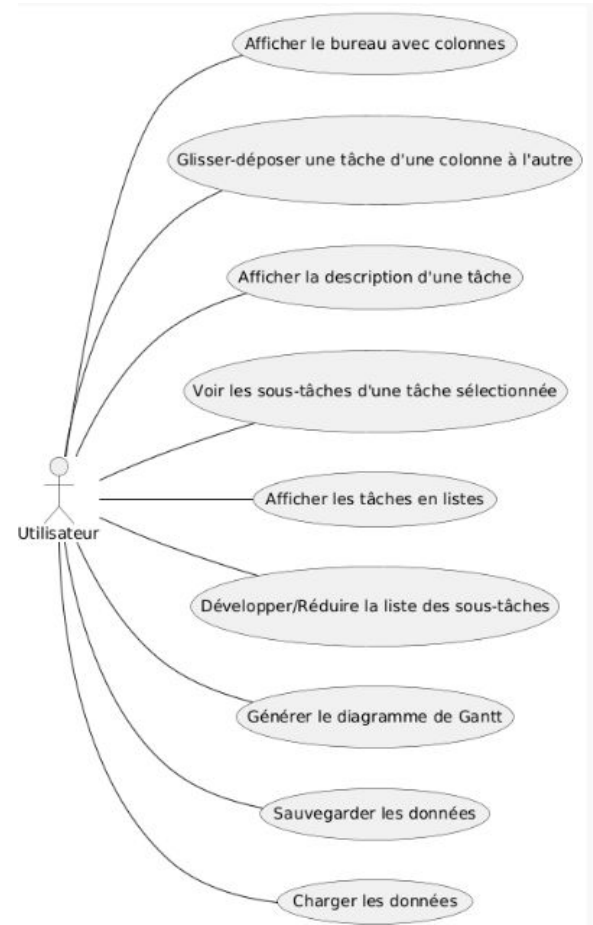
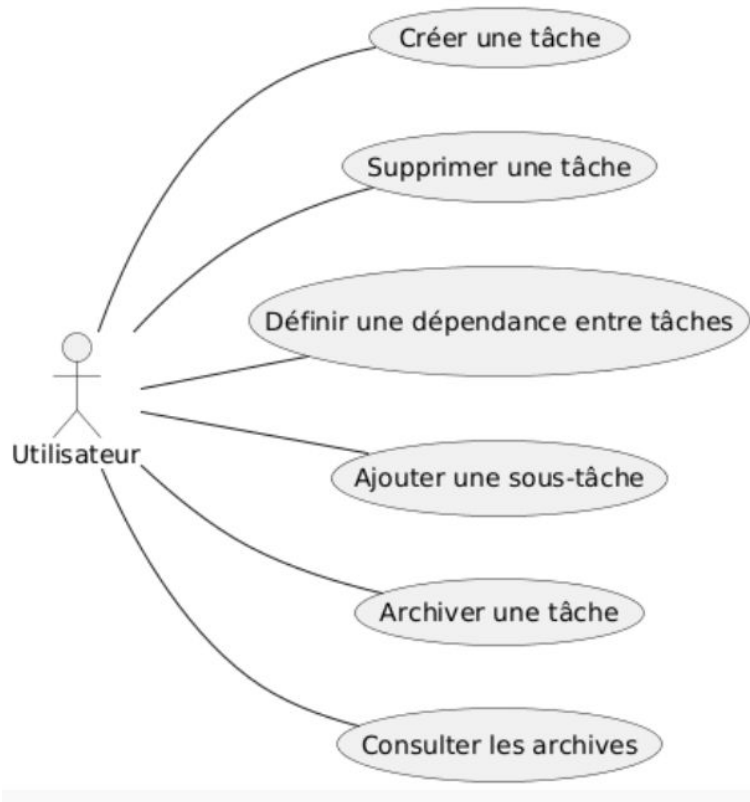
Présentation Analyse SAE 3.01

Coquin Arnaud
Meziani Fahim
Valeur–Maselli Marius
Khalfouni Fady

Fonctionnalités à implémenter

- Créer une nouvelle tâche.
- Modifier une tâche.
- Supprimer une tâche.
- Ajouter une description à une tâche.
- Définir une date de début et une date de fin pour chaque tâche.
- Archiver une tâche terminée.
- Définir une tâche comme sous-tâche d'une autre.
- Définir une dépendance entre deux tâches.
- Gérer le statut d'une tâche.
- Attribuer un niveau de priorité.
- Pour une tâche principale, calculer un pourcentage d'avancement basé sur le nombre de ses sous-tâches terminées.
- Créer des colonnes.
- Supprimer une colonne.
- Générer un diagramme de Gantt.
- Visualiser l'ensemble des tâches sous forme d'un bureau.
- Déplier/Replier une tâche.

Diagramme de cas d'utilisation



Description textuelle

Cas : Créer une tâche

Déroulement normal :

L'utilisateur sélectionne l'option pour créer une nouvelle tâche.

Le système répond en affichant le formulaire de saisie.

L'utilisateur remplit les informations de la nouvelle tâche, telles que le nom, la description, la date d'échéance puis valide la création.

Le système enregistre la nouvelle tâche.

Variantes :

- Variante A, étape 3 : Création avec dépendance ->

L'utilisateur sélectionne l'option d'ajout de dépendance à la création de la tâche après les autres informations.

Le système affiche la liste de tâches existantes.

L'utilisateur choisit les tâches "prédécesseurs" nécessaires, et valide.

Le système met à jour la structure de dépendance.

- Variante B, étape 3 : Nom manquantes ->

L'utilisateur valide la création sans remplir le nom de la tâche.

Le système refuse.

Diagramme de séquence

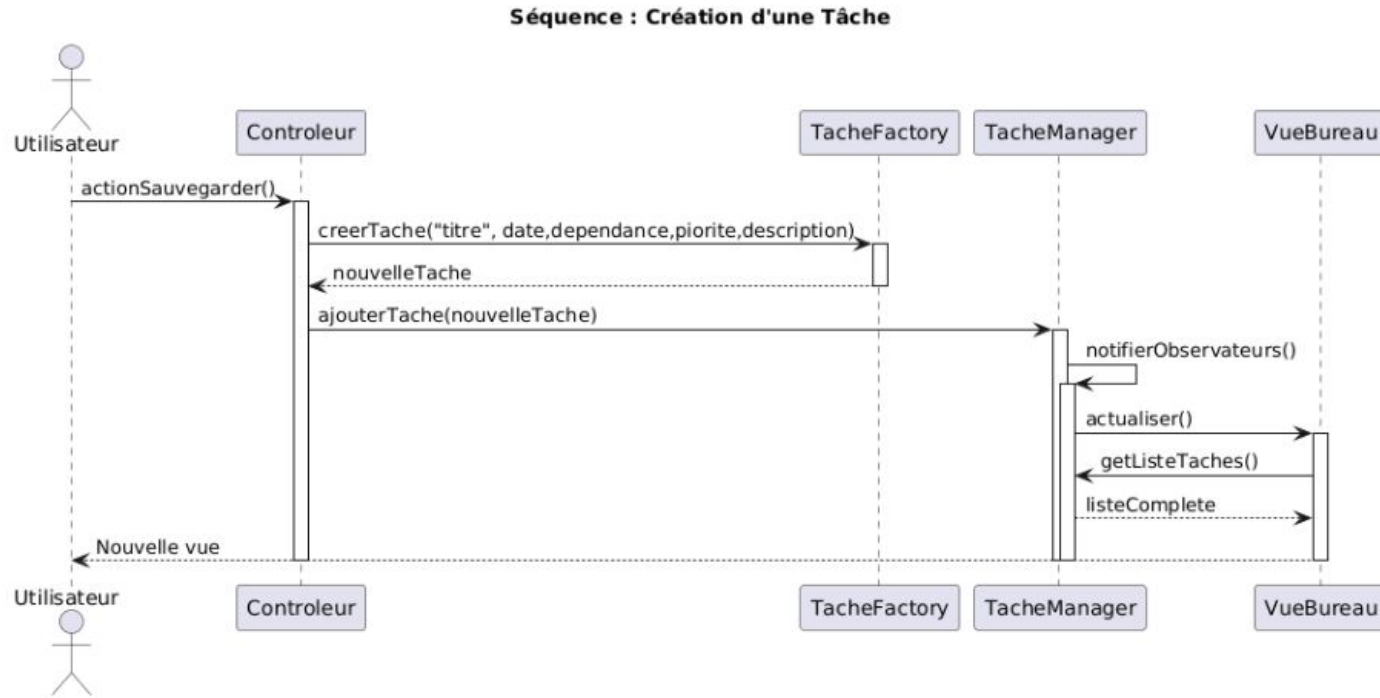


Diagramme d'activité de l'application

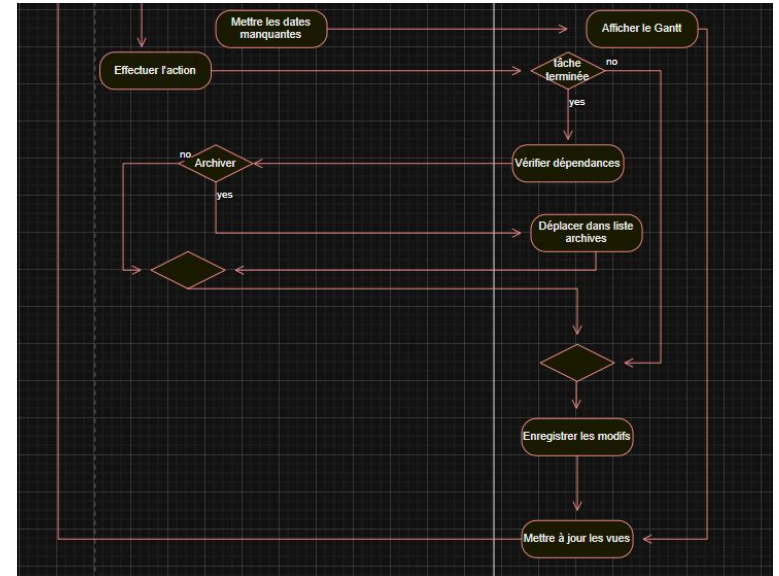
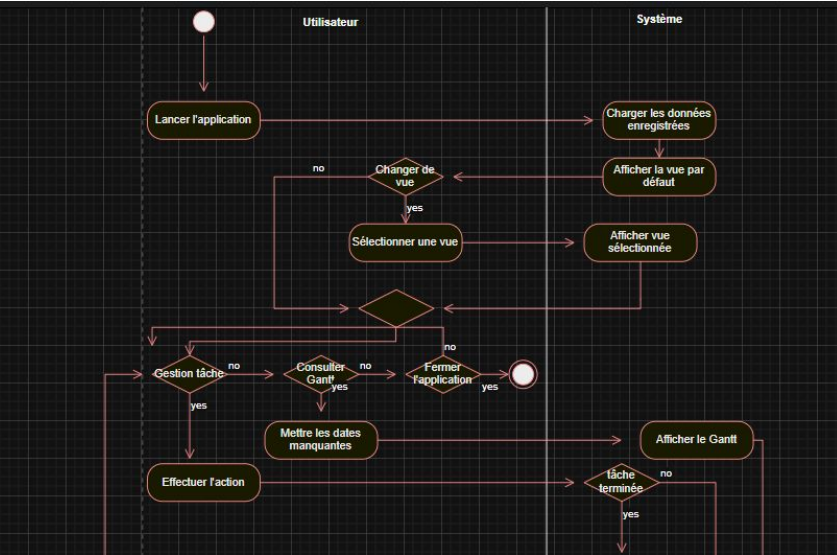
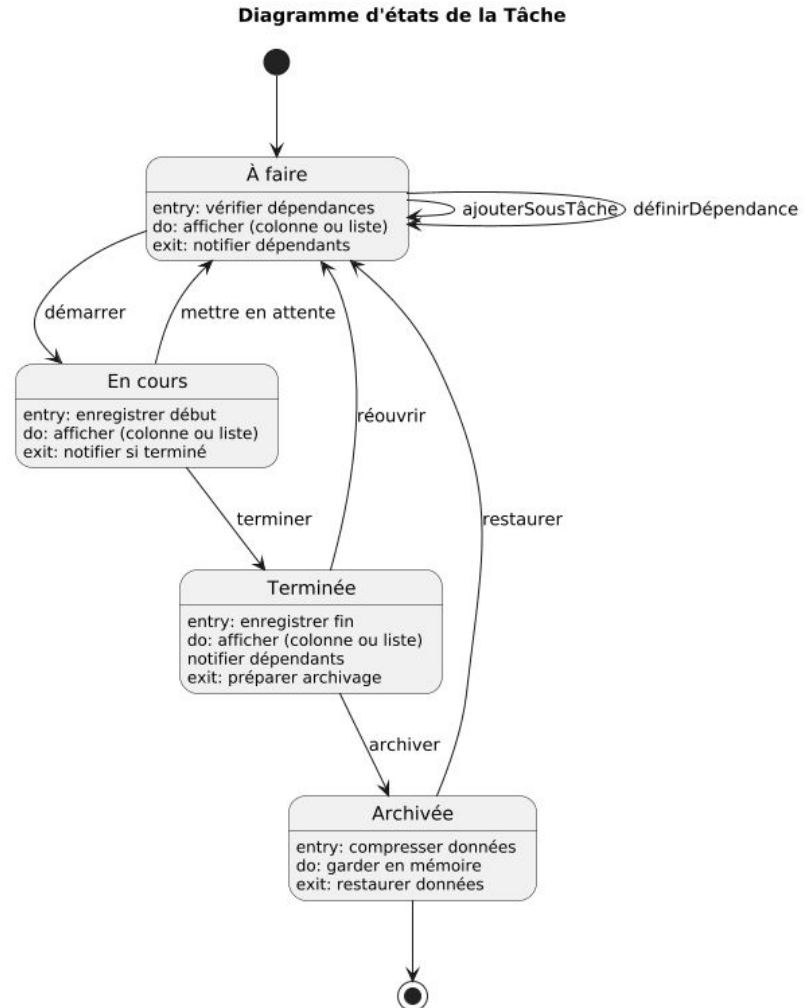


Diagramme d'état



Les patrons à utiliser

Patron architecture:

L'Architecture MVC

il sert à diviser le code de l'application en trois parties distinctes :

1. Le Modèle : Les données
2. La Vue : Ce que l'utilisateur voit
3. Le Contrôleur : Le chef d'orchestre qui fait le lien

C'est une exigence du sujet de la SAE. il est très utilisé dans javafx pour une application qui évolue change et qui est optimisé et elle facilite le travail car on peut travailler sur une partie pendant que quelqu'un travaille sur une autre

Liste des patrons de conception que nous allons utiliser et pourquoi :

Patron composite:

Il permet de créer une structure en arbre où un élément peut contenir d'autres éléments du même type.

C'est la solution technique idéale pour répondre à l'exigence du sujet qui demande de créer des tâches incluses dans une autre tâche. Grâce à lui, on pourra calculer l'avancement d'une "tâche mère" simplement en regardant ses "tâches filles" et donc avoir notre diagramme de gantt.

Patron Observateur

Il permet d'actualiser les vues.

Sachant qu'on utilise un modèle MVC, il est obligatoire d'être là comme nous avons plusieurs vues il faut que si l'utilisateur modifie une tâche dans la liste, elle se mette à jour immédiatement dans les vues sans avoir besoin de rafraîchir l'application.

Le Patron Stratégie

Il permet d'avoir plusieurs façons différentes de faire la même action et de changer de méthode selon le choix de l'utilisateur.

Notre application doit afficher les mêmes tâches sous trois formes très différentes : Vue Bureau, Vue Liste ou Vue Gantt. Ce patron permet de passer d'une vue à l'autre proprement.

Le Patron Fabrique

Il permet de créer des objets facilement de manière optimale

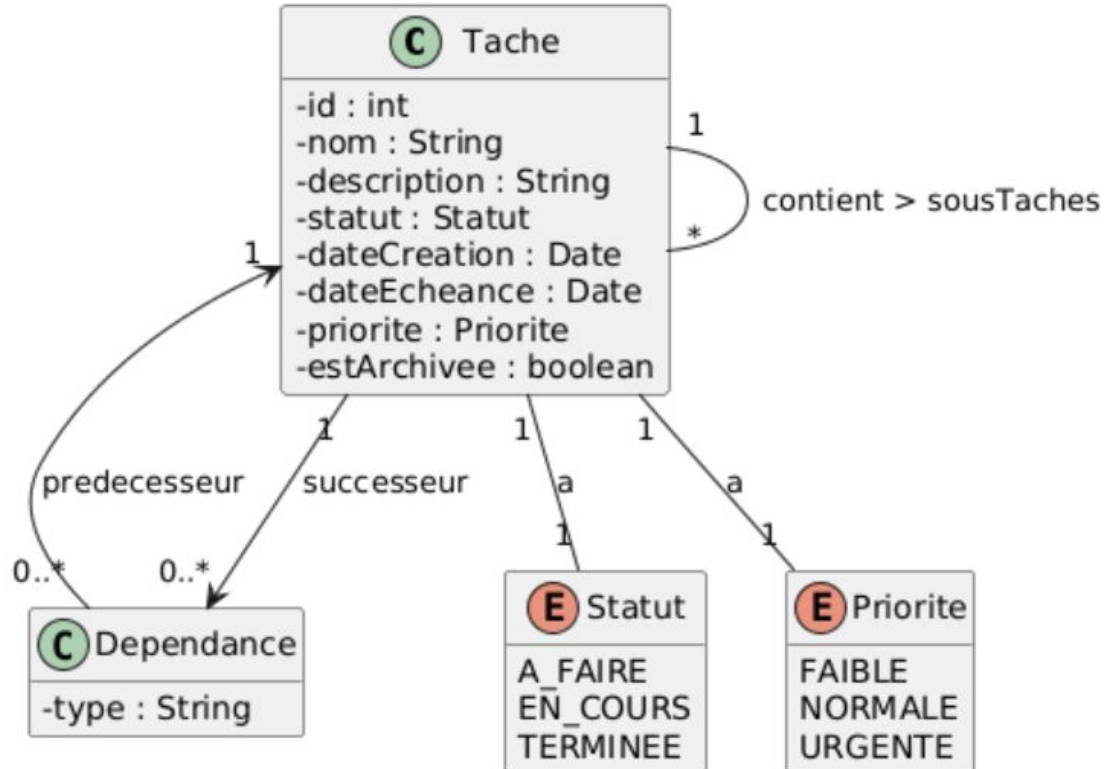
Pour éviter les erreurs à la création. Quand on clique sur "Nouvelle Tâche", le patron garantit que la tâche aura bien un identifiant unique, une date de création et un état "À faire" par défaut et il créera la tâche selon la date de dépendance au niveau de priorité.

Le Patron Singleton

Il garantit qu'une classe n'existe qu'en un seul exemplaire unique dans tout le programme et permet d'y accéder de partout.

Pour gérer notre "Gestionnaire de Tâches". Il faut qu'il n'y ait qu'une seule donnée en mémoire pour toute l'application. Cela évite d'avoir des versions différentes des tâches et qu'une seule instance existe.

Diagramme de classe



[Vue Bureau](#)[Vue Liste](#)[Diagramme de
Gantt](#)[Archives](#)[Paramètres](#)

A Faire

Conception UML

 15/12/2025

IMPOTANT !

Faire l'interface

bloquer par indépendance: conception

 26/12/2025

Moyenne !

En Cours

Diagramme de classe

 15/12/2025

EN Cours!

Terminées

Liste des fonctionnalités

Fin le 18 /12/2025

Vue Bureau

Vue Liste

Diagramme de
Gantt

Archives

Paramètres

Nom de la tache	Date limite	Priorité	Etat
▼ Conception UML	18 janvier	IMPOTANT !	En Cours
1.Faire Diagramme de classe	10 janvier	Moyenne !	Bloquer
2Faire Diagramme d'activité	8 janvier	Basse !	Terminer
► Préparer la soutenance	10 fevrier	Moyenne !	A faire

Vue Bureau

Vue Liste

Diagramme de
Gantt

Archives

Paramètres

A Faire

Conception UML

15/12/2025

IMPRO

Faire l'interface

bloquer par indépendance: conception

26/12/2025

Moye

Terminées

Liste des fonctionnalités

Fini le 18 /12/2025

Créer une nouvelle tâche

Titre de la tache

Titre

Date limite

jj/mm/aaaa

Priorite

sélecteur

Basse

Moyenne

Haute

Dépendance

sélecteur

Tache 1

Tache 2

tache 3

Description

Description

Annuler

SAUVEGARDER

Trello Archives

Vue Bureau

Vue Liste

Diagramme de
Gantt

Archives

Paramètres

Nom de la tâche	Date origine	Date ajout	Action	
► Conception UML	18 janvier	10 février	Restaurer	Supprimer
► Préparer la soutenance	10 février	18 février	Restaurer	Supprimer

[Vue Bureau](#)[Vue Liste](#)[Diagramme de Gantt](#)[Archives](#)[Paramètres](#)